Technical Concepts Study Guide: SciNet Scientific Applications & AI Analyst

Companion to: `scinet-uoft-scientific-applications-ai-analyst.md` (strategy and behavioural prep)
Prepared: 2026-10-14
How to use this: each section gives the concept, the numbers or commands worth memorizing, and a **"Say it like this"** line, which is the 15-second answer an interviewer wants. **Your anchor** lines tie a concept to something you've actually done, so you answer from experience, not theory. Items marked *(verify)* are my best understanding but not confirmed from an official source. Check them on docs.scinet.utoronto.ca before you repeat them.

Priority if time is short: §1 (cluster usage) → §3–§5 (parallel models, MPI, OpenMP) → §11 (distributed training) → §12 (LLMs) → §2 (hardware numbers) → §17 (scenario drills) → §18 (quiz). Skim the rest.

Contents

1. Using an HPC cluster (the daily reality of SciNet users)
2. Computer hardware and networking
3. Parallel programming models
4. MPI essentials
5. OpenMP essentials
6. CUDA and GPU programming basics
7. Compilers, optimization and debugging
8. Fortran: read and modify it with confidence
9. Python for performance and data
10. Machine learning: helping users design, train and *validate* models
11. Distributed deep learning (central to the new AI system)
12. LLMs: inference, fine-tuning and "AI-assisted research"
13. Production, monitoring and software engineering for science
14. Visualization (asset)
15. Teaching and training (an asset they listed, and half the role)
16. Numbers worth memorizing
17. Scenario drills (practise aloud: diagnosis → fix → prevention)
18. Self-quiz (cover the answers, then check)
19. Honest-gap language (keep it ready)
20. Sources and further reading

1. Using an HPC cluster (the daily reality of SciNet users)

1.1 Anatomy of a cluster

- **Login nodes:** where users SSH in, edit, compile, submit jobs. *Never run heavy compute here.* That's the #1 new-user mistake, and analysts deal with it constantly.
- **Compute nodes:** reached only through the scheduler. Often **no internet access**, so models, datasets and pip packages must be fetched on a login or data-transfer node first.
- **Data-transfer nodes (DTN):** for big transfers. **Globus** for large or reliable transfers; `rsync` / `scp` for small ones.
- **Shared filesystems:** typically `$HOME` (small, backed up), `$PROJECT` (group, backed up or persistent), `$SCRATCH` (large, fast, **purged** after inactivity, not backed up). Check SciNet's exact quotas and purge policy *(verify)*.
- **Node-local disk** (NVMe/ `$SLURM_TMPDIR`-style, if provided): fastest I/O for a job's lifetime. Stage data in at job start, copy results out at the end.

1.2 Trillium and SciNet systems (numbers to know)

System	What	Notes
Trillium (Aug 2025)	241,056 CPU cores + 252 GPUs	Replaced Niagara (CPU) and Mist (GPU). The arithmetic fits **1,224 CPU nodes × 192 cores** (2× 96-core AMD EPYC) + **63 GPU nodes × 4 GPUs** with 96 cores each *(verify)*. GPUs believed to be **NVIDIA H100 SXM 80 GB** *(verify)*. Storage: VAST all-flash *(verify)*.
New AI system	$42.5M, Sovereign AI Compute Strategy via the Alliance's National AI Compute – Rapid Deployment	Delivery targeted H1 2026. Name and GPU model not public. **Ask them.**
Balam	41× NVIDIA A100	Run for the Acceleration Consortium (U of T, AI for materials and chemistry discovery)
Rouge	AMD GPU cluster	AMD donation; ROCm, not CUDA
Teach	Teaching cluster	Used in courses and workshops
S4H	Secure enclave	Sensitive or health data

1.3 Slurm (the scheduler): commands and concepts

Command	Purpose
`sbatch job.sh`	Submit a batch script
`salloc --gpus-per-node=1 --time=1:00:00`	Interactive allocation (debugging)
`srun`	Launch parallel tasks inside an allocation (replaces `mpirun` on most clusters)
`squeue -u $USER` / `squeue --start`	Queue state / estimated start
`scancel <jobid>`	Cancel
`sacct -j <jobid> --format=JobID,Elapsed,MaxRSS,State`	Accounting after the job
`seff <jobid>`	Efficiency summary (CPU and memory used vs requested)
`sinfo`	Partitions and node states
`scontrol show job <jobid>`	Full job detail, including the reason it's pending

Key directives: `--nodes`, `--ntasks-per-node`, `--cpus-per-task`, `--gpus-per-node` (or `--gres=gpu:4`), `--mem`, `--time`, `--account=<allocation>`, `--partition`, `--array=1-100%10` (a job array with at most 10 running at once), `--dependency=afterok:<jobid>` (pipelines).

Concepts the panel may probe:

- **Fair-share:** priority falls as your group uses its share. Allocations come through the Alliance's **Resource Allocation Competition (RAC)**; everyone else gets a default (opportunistic) allocation.
- **Backfill:** the scheduler fits short jobs into gaps before big jobs start. So **requesting an accurate (shorter) wall time makes jobs start sooner.** Great user tip.
- **Whole-node vs per-GPU allocation:** some systems allocate CPU work by whole node. Check how SciNet allocates GPUs *(verify)*.
- **Job arrays** for parameter sweeps or hyperparameter search: one script, `$SLURM_ARRAY_TASK_ID` selects the config.

- **Checkpoint/restart:** jobs have wall-time limits (often 24 h). Long training must save and resume. Chain jobs with `--dependency`.

```
#!/bin/bash
#SBATCH --job-name=hpsweep
#SBATCH --array=0-23%8            # 24 configs, at most 8 at once
#SBATCH --gpus-per-node=1
#SBATCH --cpus-per-task=12
#SBATCH --time=03:00:00
#SBATCH --output=logs/%x-%A_%a.out
module load python cuda           # exact names: module spider
source ~/venvs/ml/bin/activate
python train.py --config configs/cfg_${SLURM_ARRAY_TASK_ID}.yaml
```

1.4 Software environment

- **Lmod modules:** `module avail`, `module spider <name>` (search, including versions and dependencies), `module load gcc openmpi`, `module list`, `module purge`. On Alliance systems the software stack is delivered through **CVMFS** and versioned with **StdEnv** *(SciNet may differ; verify)*.
- **Python on clusters:** `virtualenv` + `pip install --no-index` using the Alliance's pre-built wheels. Conda is discouraged on Alliance systems: thousands of small files hit filesystem quotas and slow the metadata servers, and its bundled libraries conflict with the system MPI and CUDA.
- **Containers: Apptainer** (formerly Singularity), not Docker. It runs rootless as the user, uses a single image file (SIF), which is filesystem-friendly, and has GPU passthrough with `--nv`. You can convert Docker images: `apptainer build pytorch.sif docker://nvcr.io/nvidia/pytorch:24.xx-py3`, then `apptainer exec --nv pytorch.sif python train.py`.
- **Jupyter:** JupyterHub or Open OnDemand for interactive work. Good for teaching and exploration; batch jobs for production.
- **MFA and SSH keys:** Alliance clusters require multifactor authentication. Users should use SSH keys, never share accounts.

Say it like this: "*The pattern I'd teach every new user: log in, load modules, build a venv once, keep code in home, data on scratch or project, stage hot data to node-local disk, submit with an honest wall time, checkpoint, then run* `seff` *to see what you actually used.*"

---

2. Computer hardware and networking

2.1 CPU and memory

- **Cores, SMT** (hyper-threading; often disabled in HPC), **clock**, **SIMD width** (AVX2 = 256-bit = 4 doubles; AVX-512 = 8 doubles per instruction).
- **Cache hierarchy:** L1 (~32–48 KB/core, ~1 ns), L2 (~1 MB/core, ~3–5 ns), L3 (shared, tens to hundreds of MB, ~10–20 ns), DRAM (~80–100 ns). **Memory bandwidth**, not FLOPs, limits most scientific codes.
- **NUMA:** a dual-socket node has two (or more) memory domains. Remote-socket memory is slower. **First-touch policy:** a page lives on the NUMA node of the thread that first writes it, so initialize arrays in parallel with the same thread layout you compute with. Pin threads (`OMP_PROC_BIND=close`, `OMP_PLACES=cores`; `srun --cpu-bind`).
- **Spatial locality:** cache lines are 64 bytes. Stride-1 access wins. Row-major (C) vs column-major (Fortran) decides which loop goes innermost.

2.2 GPU architecture (NVIDIA)

- **SM (streaming multiprocessor):** executes **warps of 32 threads** in lockstep (SIMT). H100 SXM has 132 SMs; A100 has 108.
- **Memory hierarchy:** registers (fastest, per thread) → shared memory / L1 (per SM, ~228 KB on H100, programmer-managed) → L2 (50 MB H100, 40 MB A100) → **HBM** (global memory).
- **Tensor Cores:** matrix-multiply units for FP16/BF16/TF32/FP8/INT8. This is where the advertised deep-learning FLOPs come from.
- **Warp divergence:** an `if` that splits a warp serializes both paths. **Coalescing:** adjacent threads reading adjacent addresses means one wide memory transaction. **Occupancy:** enough active warps to hide memory latency.

Spec (approx.)	A100 80GB SXM	H100 80GB SXM
HBM	80 GB HBM2e, ~2.0 TB/s	80 GB HBM3, ~3.35 TB/s
BF16/FP16 dense tensor	~312 TFLOPS	~989 TFLOPS
FP8 dense	—	~1,979 TFLOPS
FP64 (non-tensor)	~9.7 TFLOPS	~34 TFLOPS
NVLink per GPU	600 GB/s (NVLink 3)	900 GB/s (NVLink 4)
TDP	400 W	up to 700 W

(Spec sheets quote 2× these with "sparsity". Use the dense numbers.)

2.3 Interconnects (fastest to slowest)

Link	Typical bandwidth	Use
NVLink/NVSwitch	600–900 GB/s per GPU	GPU↔GPU inside a node; tensor parallelism lives here
PCIe Gen4 / Gen5 x16	~32 / ~64 GB/s per direction	CPU↔GPU, NIC↔GPU
InfiniBand HDR / NDR	200 / 400 **Gb/s** = 25 / 50 **GB/s** per port	Node↔node; MPI and NCCL across nodes; ~1 µs latency
Ethernet 100 GbE	12.5 GB/s, higher latency	Storage, management; RoCE adds RDMA over Ethernet

- **Gb vs GB:** divide bits by 8. Interviewers like catching this.
- **RDMA:** remote direct memory access. The NIC moves data between hosts' memory without CPU involvement (no kernel copies) → low latency.
- **GPUDirect RDMA:** NIC reads and writes GPU memory directly (no bounce through host RAM). **GPUDirect Storage:** NVMe or parallel FS → GPU directly.
- **Topologies: fat-tree** (full bisection bandwidth if non-blocking; "2:1 oversubscription" means half), **dragonfly** (fewer long cables, used at very large scale). **Bisection bandwidth** limits all-to-all patterns.
- **Rail-optimized GPU networks:** each GPU in a node gets its own NIC (e.g. 4 GPUs → 4 NDR ports) so collectives don't bottleneck on one link.

Your anchor: MIRaaS (IEEE IS² 2026) is a latency-characterization study of offloading computation over UDP. You've measured network latency budgets end to end. "*Same discipline, different scale: there it was Wi-Fi and UDP at millisecond budgets; here it's InfiniBand and NCCL at microseconds.*"

2.4 Storage and I/O

- **Parallel filesystems:** Lustre (metadata server MDS + object storage targets OSTs; striping), IBM Spectrum Scale/GPFS, **VAST** (all-flash, NFS-based, disaggregated). Bandwidth scales with servers; **metadata ops** (open, stat, ls) are the scarce resource.
- **The small-files problem:** a million 50 KB images means a million metadata ops per epoch → slow for everyone. Fix: **pack** into tar shards (WebDataset), HDF5, Zarr, Parquet, LMDB; read sequentially; stage to local disk.
- **Formats:** CSV (human-readable, slow, large) → **Parquet/Arrow** (columnar, compressed, fast column scans) → **HDF5/NetCDF** (n-d arrays with metadata, the standard in physics and climate; parallel HDF5 via MPI-IO) → **Zarr** (chunked, cloud and object-store friendly).
- **Checkpoints:** few large files, written asynchronously, keep the last N, and never write every step.
- **HPSS/tape:** archival, not working storage.

---

3. Parallel programming models

3.1 The taxonomy (memorize this one)

Model	Memory	Scope	Typical tool
Vectorization (SIMD)	—	one core, many data lanes	compiler auto-vectorization, intrinsics
Shared-memory threads	shared	one node	**OpenMP**, pthreads, TBB
Distributed memory (message passing)	private per process	many nodes	**MPI**
Accelerators	separate device memory	GPU	**CUDA**, HIP (AMD), OpenACC, OpenMP offload, SYCL
Hybrid	mixed	cluster	MPI between nodes + OpenMP or CUDA within

Say it like this: "*OpenMP is several cooks in one kitchen sharing a fridge. MPI is several kitchens that phone each other with ingredients. A GPU is a kitchen of thousands of line cooks doing simple steps in lockstep. Large codes mix all three. And multi-GPU deep learning is really MPI-style collectives: the gradient all-reduce.*"

Also know **Flynn's taxonomy:** SISD, SIMD, MIMD. Clusters are MIMD; GPUs are SIMT (a SIMD variant). The JD's phrase "serial, vector, and parallel systems" maps to: a single core, SIMD/vector units (and historically Cray vector machines), and multicore/cluster.

3.2 Scaling laws

- **Amdahl (strong scaling, fixed problem):** speedup $S(N) = 1 / ((1-p) + p/N)$. If $p = 0.9$ and $N = 8$: $S = 1/(0.1 + 0.1125) \approx$ **4.7**. Maximum as $N \to \infty = 1/(1-p) =$ **10**.
- **Gustafson (weak scaling, problem grows with N):** $S(N) = (1-p) + p \cdot N$. That's why big machines run big problems efficiently.
- **Efficiency:** strong $E = T_1 / (N \cdot T_N)$; weak $E = T_1 / T_N$ (ideal = 1).
- **Scaling test:** before granting or recommending large allocations, run 1, 2, 4, 8… nodes and plot. Stop where efficiency drops below ~70–80%. SciNet and the Alliance often require scaling evidence in allocation proposals.

3.3 Load balance, communication, synchronization

- Total time ≈ compute + communication + idle (imbalance) + I/O.
- **Latency-bandwidth (α–β) model:** message time ≈ α + n/β. Many small messages are latency-bound; aggregate them.
- **Overlap** communication with computation (non-blocking MPI; DDP's bucketed all-reduce during backward).
- **Surface-to-volume:** domain decomposition. Communication scales with boundary area, compute with volume, so bigger subdomains per rank mean better efficiency.

---

4. MPI essentials

4.1 Core concepts

- **Ranks** (process IDs) inside a **communicator** (`MPI_COMM_WORLD`); `size` = number of ranks.
- **Point-to-point:** `MPI_Send` / `MPI_Recv` (blocking), `MPI_Isend` / `MPI_Irecv` + `MPI_Wait` / `MPI_Waitall` (non-blocking), `MPI_Sendrecv` (deadlock-safe exchange).
- **Collectives:** `Bcast`, `Reduce`, **`Allreduce`**, `Scatter`, `Gather`, `Allgather`, `Alltoall`, `Barrier`, `Reduce_scatter`.
- **Deadlock classic:** two ranks each call a blocking `MPI_Send` to the other before receiving. A large message won't be buffered, so both wait forever. Fixes: order the calls (even sends first, odd receives first), use `MPI_Sendrecv`, or use non-blocking calls.
- **Halo/ghost-cell exchange:** domain-decomposed PDE solvers swap boundary layers with neighbours each step.
- **MPI-IO / parallel HDF5:** many ranks write one shared file collectively.
- **Implementations:** Open MPI, MPICH, Intel MPI, MVAPICH. On Slurm use `srun ./app` (or `mpirun` per site docs).

4.2 Code: compute π by numerical integration (C)

```c
#include <mpi.h>
#include <stdio.h>

int main(int argc, char **argv) {
    MPI_Init(&argc, &argv);
    int rank, size;
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);

    const long n = 100000000;
    const double h = 1.0 / n;
    double local = 0.0;
    for (long i = rank; i < n; i += size) {      /* cyclic distribution */
        double x = h * (i + 0.5);
        local += 4.0 / (1.0 + x * x);
    }
    double pi = 0.0;
    MPI_Reduce(&local, &pi, 1, MPI_DOUBLE, MPI_SUM, 0, MPI_COMM_WORLD);
    if (rank == 0) printf("pi ~ %.12f\n", pi * h);
    MPI_Finalize();
    return 0;
}
```

`mpicc -O3 pi.c -o pi` → `srun -n 8 ./pi` (or `mpirun -n 8 ./pi` locally).

4.3 Code: mpi4py all-reduce (the bridge to deep learning)

```python
from mpi4py import MPI
import numpy as np

comm = MPI.COMM_WORLD
rank, size = comm.Get_rank(), comm.Get_size()
local = np.full(4, rank, dtype="d")      # pretend: this rank's gradients
total = np.empty_like(local)
comm.Allreduce(local, total, op=MPI.SUM)  # every rank gets the sum
if rank == 0:
    print(total / size)                   # averaged "gradient"
```

Say it like this: "*Data-parallel training is exactly this: each GPU computes gradients on its shard, an all-reduce averages them, and every replica takes the same optimizer step. NCCL is a GPU-optimized collective library doing what MPI_Allreduce does.*"

---

5. OpenMP essentials

- `#pragma omp parallel for` splits loop iterations across threads. Set `OMP_NUM_THREADS` to match `--cpus-per-task`.
- **Data-sharing:** `shared` (default for most variables), `private`, `firstprivate`; loop indices are private.
- **`reduction(+:sum)`** avoids races on accumulators.
- **Race condition:** threads write the same variable without synchronization. Fix with reduction, `atomic`, `critical` (slowest), or restructuring.
- **False sharing:** threads write different variables on the **same 64-byte cache line**, and the line ping-pongs between cores. Fix with padding or per-thread local accumulators.
- **Scheduling:** `static` (equal chunks, lowest overhead), `dynamic` (work-stealing for uneven iterations), `guided`.
- **Offload:** `#pragma omp target teams distribute parallel for` runs on a GPU. OpenACC is the directive alternative, popular for porting Fortran codes to GPUs.

```c
double dot = 0.0;
#pragma omp parallel for reduction(+:dot) schedule(static)
for (long i = 0; i < n; i++)
    dot += a[i] * b[i];
```

Compile: `gcc -O3 -fopenmp` / `gfortran -O3 -fopenmp` / `nvc -mp`.

---

6. CUDA and GPU programming basics

6.1 Programming model

- **Kernel** (`__global__`) launched over a **grid of blocks of threads**. Global index = `blockIdx.x * blockDim.x + threadIdx.x`.
- **Host vs device memory:** `cudaMalloc`, `cudaMemcpy` (PCIe is slow relative to HBM, so **minimize transfers** and keep data on the GPU), unified/managed memory (`cudaMallocManaged`) for convenience.
- **Streams:** asynchronous queues that overlap copies with kernels.

- **Libraries before hand-written kernels:** cuBLAS, cuDNN, cuFFT, cuSPARSE, cuSOLVER, NCCL, Thrust, CUTLASS. In Python: **CuPy** (NumPy on GPU), **Numba CUDA**, **PyTorch**, **JAX**, RAPIDS (cuDF, cuML). **Triton** (OpenAI's language, not NVIDIA's Triton Inference Server) for custom fused kernels in Python.
- **AMD:** HIP/ROCm (CUDA-like; `hipify` ports code). Relevant to the Rouge cluster.

```
__global__ void saxpy(int n, float a, const float *x, float *y) {
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    if (i < n) y[i] = a * x[i] + y[i];          // bounds check for the last block
}
// launch: saxpy<<<(n + 255) / 256, 256>>>(n, 2.0f, d_x, d_y);
```

6.2 Performance concepts

- **Roofline model:** attainable FLOP/s = min(peak FLOP/s, arithmetic intensity × memory bandwidth). **Arithmetic intensity** = FLOPs / bytes moved. H100 BF16 ridge point ≈ 989 TFLOPS / 3.35 TB/s ≈ **~300 FLOP/byte**. Below that you're **memory-bound** (element-wise ops, LayerNorm, LLM decode); above it **compute-bound** (large matmuls).
- **Kernel fusion** (FlashAttention, `torch.compile`, Triton) cuts memory traffic for memory-bound chains of ops.
- **Low GPU utilization causes, in order of frequency:** input pipeline (CPU decode, slow filesystem), tiny batch or kernels (launch overhead → use CUDA Graphs or `torch.compile`), host↔device copies, synchronization (`.item()`, `print(tensor)` in the loop), communication.
- **Tools:** `nvidia-smi` / `nvidia-smi dmon` / `nvtop` (quick look), **Nsight Systems** `nsys profile` (timeline: CPU, GPU, NCCL, I/O), **Nsight Compute** `ncu` (per-kernel deep dive), `torch.profiler` + TensorBoard, **DCGM** (cluster-wide GPU metrics into Prometheus/Grafana).

Your anchor: you profiled CPU budgets on ARM (14 ms at 31.4% CPU on an RPi4). "*Measure first, then optimize, at any scale. On the Pi it was perf counters and per-feature latency in Nebula; on H100s it's Nsight Systems.*"

---

7. Compilers, optimization and debugging

7.1 Compilers you'll meet

GNU (`gcc`, `g++`, `gfortran`) · Intel oneAPI (`icx`, `icpx`, `ifx`) · NVIDIA HPC SDK (`nvc`, `nvc++`, `nvfortran`; OpenACC and CUDA Fortran) · LLVM (`clang`, `flang`) · AMD AOCC (tuned for EPYC). MPI wrappers: `mpicc`, `mpicxx`, `mpif90`. They call the underlying compiler with MPI include and link flags.

7.2 Flags

Flag	Meaning
`-O0 -g`	Debug build
`-O2`	Safe standard optimization
`-O3`	Aggressive: vectorization, unrolling, inlining
`-march=native` / `-march=znver4`	Use this CPU's instruction set (AVX-512 on Zen 4/5). Binary may not run on older CPUs.
`-ffast-math`	Relaxes IEEE rules (reassociation, no NaN/Inf checks). Faster, but **can change results**; validate first.
`-fopenmp`	Enable OpenMP

Flag	Meaning
`-fopt-info-vec-missed` (GCC) / `-qopt-report` (Intel)	Why a loop didn't vectorize
`-fcheck=all` (gfortran)	Runtime bounds and argument checks (debugging)
`-fsanitize=address,undefined`	Memory-error and undefined-behaviour detection

7.3 Why loops don't vectorize

Loop-carried dependencies (`a[i] = a[i-1] + …`), possible **pointer aliasing** in C (fix: `restrict`), function calls inside the loop, non-unit stride or indirect indexing (`a[idx[i]]`), branches. Fortran's no-aliasing rule is one reason Fortran vectorizes well.

7.4 Optimization workflow (say this order)

1. **Correct first,** with a regression test and tolerance.
2. **Profile:** `gprof`, `perf`, Intel VTune, `py-spy`, Nsight. Find the 10% of code taking 90% of the time.
3. **Algorithm and complexity** before micro-tuning (your RNN vs DTW case: an algorithmic change gave 74×).
4. **Libraries:** BLAS/LAPACK (OpenBLAS, MKL, BLIS; FlexiBLAS on Alliance systems lets you switch backends), FFTW, PETSc, Eigen.
5. **Memory layout and locality,** then **vectorize,** then **parallelize** (threads → MPI → GPU).
6. **Re-measure;** check the numerics still pass.

7.5 Building and linking

`make`, **CMake** (`cmake -B build -DCMAKE_BUILD_TYPE=Release && cmake --build build -j`), static (`.a`) vs shared (`.so`) libraries, `LD_LIBRARY_PATH`, `ldd ./app` (which libraries it resolves), `rpath`. "Works on my laptop, fails on the cluster" is usually a module or library mismatch.

7.6 Debugging

`gdb` (backtrace from a core dump: `ulimit -c unlimited`), `valgrind --tool=memcheck` (leaks, invalid reads), `cuda-gdb`, `compute-sanitizer` (CUDA memory errors), parallel debuggers (DDT/Linaro Forge, TotalView). For MPI hangs: attach `gdb` to the stuck rank, or print rank-tagged logs.

Your anchor: Nebula is C++17 with CMake, FFTW3 and libsndfile, plus per-feature latency benchmarking. "*I've built and tuned numerical C++ against FFTW with CMake; the build-and-profile loop is the same on a cluster, with modules on top.*"

---

8. Fortran: read and modify it with confidence

8.1 What a C/Python programmer needs to know

- **1-based** indexing (bounds can be custom: `real :: a(0:n)`); **column-major** storage → **innermost loop over the first index**.
- `implicit none` always (otherwise variables starting i–n are implicitly integer).
- Types: `integer`, `real(dp)` (with `dp = kind(1.0d0)` or `real64` from `iso_fortran_env`), `complex(dp)`, `logical`, `character(len=*)`.

- **Array syntax:** `c = a + b`, `x(2:n-1)`, `sum(x)`, `maxval`, `matmul`, `dot_product`, `where (x > 0) y = sqrt(x)`.
- `allocatable` arrays: `allocate(x(n))`, `deallocate(x)`.
- **Procedures:** `subroutine` (no return value) vs `function`. Arguments are passed **by reference**. `intent(in/out/inout)` documents and enforces direction.
- **Modules** replace headers: `module … contains … end module`, then `use mymod`.
- Interop: `iso_c_binding` for calling C; **`f2py`** (NumPy) or `fmodpy` to call Fortran from Python.
- Standards: F77 (fixed form, `.f`, columns 7–72, `COMMON` blocks) vs F90+ (free form, `.f90`). Lots of legacy science code is F77/F90; modern is F2008/F2018 (coarrays, `do concurrent`).

8.2 Code example

```fortran
module kinds
  implicit none
  integer, parameter :: dp = kind(1.0d0)
end module kinds

program saxpy_demo
  use kinds
  implicit none
  integer, parameter :: n = 1000000
  real(dp), allocatable :: x(:), y(:)
  real(dp) :: a

  allocate(x(n), y(n))
  a = 2.0_dp
  x = 1.0_dp                ! whole-array assignment
  y = 3.0_dp
  call saxpy(n, a, x, y)
  print *, 'y(1) =', y(1), '  sum =', sum(y)
  deallocate(x, y)

contains

  subroutine saxpy(n, a, x, y)
    integer,  intent(in)    :: n
    real(dp), intent(in)    :: a, x(n)
    real(dp), intent(inout) :: y(n)
    integer :: i
    !$omp parallel do
    do i = 1, n
      y(i) = a * x(i) + y(i)
    end do
    !$omp end parallel do
  end subroutine saxpy

end program saxpy_demo
```

`gfortran -O3 -fopenmp saxpy.f90 -o saxpy && OMP_NUM_THREADS=4 ./saxpy`

Loop order matters:

```
do j = 1, n          ! columns outer
  do i = 1, n        ! rows inner -> contiguous memory, vectorizes
    c(i, j) = a(i, j) + b(i, j)
  end do
end do
```

Say it like this: "*I can read and modify Fortran: column-major, 1-based, intent attributes, modules, array syntax. I'd wrap a Fortran kernel with f2py so a researcher can drive it from Python.*"

---

9. Python for performance and data

9.1 Making Python fast (the ladder)

1. **Vectorize with NumPy** (broadcasting, ufuncs; no Python loops over elements). Know views vs copies (slicing gives a view; fancy indexing gives a copy).
2. **Better libraries:** Polars or DuckDB for large tabular data (vs pandas), SciPy sparse, scikit-learn.
3. **JIT: Numba** `@njit(parallel=True)` with `prange`; Numba CUDA for GPU kernels.
4. **Compiled extensions:** Cython, pybind11 (C++), f2py (Fortran).
5. **GPU:** CuPy (drop-in NumPy), RAPIDS cuDF/cuML, PyTorch or JAX tensors.
6. **Parallelism:** the **GIL** blocks CPU-bound threading. Use `multiprocessing` / `concurrent.futures`, `joblib`, **Dask** (parallel pandas/NumPy, task graphs, multi-node), Ray, **mpi4py**. (Python 3.13+ has an optional free-threaded build. It's experimental; don't depend on it.)
7. **Profile:** `cProfile` + snakeviz, `line_profiler`, **`py-spy`** (attach to a running job, no code changes), `scalene`, `memory_profiler`.

```
from numba import njit, prange
import numpy as np

@njit(parallel=True, fastmath=True)
def sumsq(x):
    s = 0.0
    for i in prange(x.shape[0]):   # parallel loop; Numba handles the reduction
        s += x[i] * x[i]
    return s

print(sumsq(np.random.rand(10_000_000)))
```

9.2 Data analysis workflow (JD: "cleaning, transformation, feature engineering")

- **Cleaning:** types, units, duplicates, missing values (drop vs impute: mean/median, forward-fill for time series, model-based), outliers (domain rules, IQR, robust z-score), consistent timestamps and time zones.
- **Transformation:** standardize or normalize (**fit on the training split only**), log transforms for skewed data, encoding (one-hot, target encoding with care), resampling or alignment of time series.
- **Feature engineering:** domain features (your MFCC, LFCC and spectral features are the textbook example), rolling statistics, frequency-domain features, interactions; with deep learning, learned representations or embeddings.
- **EDA:** distributions, correlations, class balance, leakage checks, then visualize before modelling.

Your anchor: you designed four public datasets (9,800+ recordings, 70+ musicians) with frozen protocols, and built Nebula (dozens of audio feature extractors). Feature engineering at depth is a real strength. Say so.

10. Machine learning: helping users design, train and *validate* models

10.1 Validation (where most research ML goes wrong; lead with this)

- **Splits:** train / validation (model selection) / **test (touched once)**.
- **Cross-validation:** k-fold, **stratified** (class balance), **group k-fold** (all samples from one patient, subject, site or instrument stay in one fold), **time-series split** (train on the past, test on the future; never shuffle).
- **Data leakage:** normalization or feature selection fit on the full dataset; random splits on temporally or spatially correlated data; duplicates across splits; target-derived features. *The single most valuable thing an analyst catches in a researcher's pipeline.*
- **Metrics:** classification: precision, recall, F1, ROC-AUC, **PR-AUC for imbalanced data**, confusion matrix, calibration (reliability curve, Brier score). Regression: MAE, RMSE, R^2. Always compare against a **simple baseline** (mean predictor, linear model, or your DTW baseline).
- **Uncertainty:** confidence intervals via bootstrap; repeat with several seeds; report mean ± std.
- **Overfitting:** bias–variance; regularization (L2/weight decay, dropout, early stopping, data augmentation); learning curves.
- **Hyperparameter search:** random search beats grid; Bayesian (Optuna); run trials as a **Slurm job array**; ASHA/Hyperband early stopping.

Your anchor: "*My JAES work compared the RNN against a DTW baseline with ablations at pattern-set sizes 1, 3 and 10, plus latency and CPU, so I'd push every user toward a baseline, a frozen test set and ablations.*" speakfrench: 6 methods, 100k trials, ROC-AUC/EER/Cohen's d.

10.2 Deep learning fundamentals (quick refresh)

- **Training loop:** forward → loss → backward (autodiff) → optimizer step. **AdamW** default; SGD with momentum for some vision work. LR schedule: **warmup + cosine decay**. Gradient clipping for stability.
- **Normalization:** BatchNorm (CNNs; batch-size sensitive, needs SyncBatchNorm across GPUs) vs **LayerNorm/RMSNorm** (transformers).
- **Architectures:** MLP, CNN (images, spectrograms), RNN/LSTM/GRU (sequences; your PhD), **Transformer** (attention: $\text{softmax}(QK^T/\sqrt{d})\cdot V$; $O(n^2)$ in sequence length; **FlashAttention** computes it exactly without materializing the $n\times n$ matrix → memory-efficient), GNNs (molecules, meshes), diffusion models (generation), U-Net (segmentation).
- **AI for science: physics-informed neural networks** (PDE residual in the loss), **neural operators** (FNO), surrogate models replacing expensive simulations, ML interatomic potentials, protein structure models (AlphaFold-class), weather foundation models. Expect users from these fields.
- **Transfer learning / fine-tuning:** start from a pretrained foundation model; freeze vs full fine-tune vs **parameter-efficient (LoRA)**.
- **Large-batch rule of thumb:** when the global batch grows k×, scale the LR roughly k× (linear) or $\sqrt{k}\times$, with warmup. Retune as needed.

11. Distributed deep learning (central to the new AI system)

11.1 Data parallelism: DDP

- One **replica per GPU**; each processes a different mini-batch shard (`DistributedSampler`); gradients are **all-reduced** (averaged) during backward in **buckets**, **overlapping** communication with the remaining backward compute; then each replica steps identically.
- **Global batch** = per-GPU batch × #GPUs × gradient-accumulation steps.
- Always prefer **DDP** over the old single-process `nn.DataParallel`.
- Launch with **`torchrun`** (sets `RANK`, `LOCAL_RANK`, `WORLD_SIZE`, rendezvous; supports elastic restarts).

```python
import os, torch, torch.distributed as dist
from torch.nn.parallel import DistributedDataParallel as DDP
from torch.utils.data import DataLoader, DistributedSampler

dist.init_process_group("nccl")                  # env vars come from torchrun
local_rank = int(os.environ["LOCAL_RANK"])
torch.cuda.set_device(local_rank)

model = DDP(build_model().cuda(), device_ids=[local_rank])
sampler = DistributedSampler(dataset)            # distinct shard per rank
loader = DataLoader(dataset, batch_size=64, sampler=sampler,
                    num_workers=8, pin_memory=True, persistent_workers=True)
opt = torch.optim.AdamW(model.parameters(), lr=3e-4)

for epoch in range(epochs):
    sampler.set_epoch(epoch)                     # new shuffle each epoch
    for x, y in loader:
        x, y = x.cuda(non_blocking=True), y.cuda(non_blocking=True)
        with torch.autocast("cuda", dtype=torch.bfloat16):
            loss = loss_fn(model(x), y)
        opt.zero_grad(set_to_none=True)
        loss.backward()                          # bucketed all-reduce overlaps here
        opt.step()
    if dist.get_rank() == 0:                     # only one rank writes
        torch.save(model.module.state_dict(), f"ckpt_{epoch}.pt")

dist.destroy_process_group()
```

11.2 Memory: where GPU memory goes during training

Per parameter with **mixed-precision Adam ≈ 16 bytes**: BF16 weights (2) + BF16 grads (2) + FP32 master weights (4) + Adam m and v (4 + 4). **Plus activations**, which scale with batch × sequence length × layers × hidden size.

Model	Weights only (BF16)	Full training states (~16 B/param)
1B	2 GB	~16 GB
7B	14 GB	~112 GB → doesn't fit on one 80 GB GPU
70B	140 GB	~1.1 TB → needs ≥14 × 80 GB GPUs just for states

Memory-saving ladder (say this in order): BF16 autocast → **activation checkpointing** (recompute in backward; ~30% more compute) → smaller micro-batch + **gradient accumulation** → **LoRA/QLoRA** → **FSDP / ZeRO** sharding → CPU/NVMe offload (last resort; slow).

11.3 Sharding and model parallelism

- **ZeRO** (DeepSpeed): stage 1 shards optimizer states; stage 2 adds gradients; stage 3 adds parameters. **PyTorch FSDP/FSDP2** ≈ ZeRO-3: parameters are all-gathered just in time for each layer's forward and backward, and gradients are **reduce-scattered**. Memory per GPU falls roughly 1/N; communication rises.
- **Tensor parallelism** (Megatron-LM): split individual matmuls across GPUs. Very communication-heavy, so **keep it inside a node on NVLink**.
- **Pipeline parallelism:** split layers into stages across GPUs or nodes; micro-batches keep stages busy; the idle **"pipeline bubble"** shrinks with more micro-batches.
- **Sequence/context parallelism:** split long sequences (ring attention). **Expert parallelism:** for Mixture-of-Experts models (all-to-all communication).
- **3D parallelism** = data × tensor × pipeline (e.g. TP = 8 within a node, PP across nodes, DP for the rest).
- **Frameworks:** PyTorch FSDP, **HF Accelerate**, PyTorch Lightning, **DeepSpeed**, **Megatron-LM / NeMo**, torchtitan, JAX/pjit.

11.4 Collectives and NCCL

- **All-reduce = reduce-scatter + all-gather.** Ring all-reduce: each GPU sends and receives ≈ 2(N−1)/N × message size → bandwidth cost is nearly independent of N (latency grows with N; **tree** algorithms help at scale).
- **NCCL** picks ring or tree and uses NVLink inside a node and InfiniBand/RDMA across nodes. Debug with `NCCL_DEBUG=INFO` (check it found IB, not a TCP socket fallback). Hangs are often one rank crashing or ranks calling collectives in different orders.
- **Worked example:** DDP of a 1B-parameter model, BF16 grads = 2 GB per step. Ring all-reduce moves ~4 GB per GPU. Over 50 GB/s (one NDR port) that's ~80 ms per step if not overlapped; over NVLink 900 GB/s, ~4 ms. *That's why topology and overlap matter.*

11.5 Mixed precision

Format	Exponent / mantissa bits	Notes
FP32	8 / 23	baseline
TF32	8 / 10	Ampere+ matmul mode for FP32 code; enable with `torch.backends.cuda.matmul.allow_tf32 = True`
FP16	5 / 10	small range → needs **loss scaling** (`GradScaler`)
BF16	8 / 7	FP32's range → no loss scaling; default on A100/H100
FP8 (E4M3 / E5M2)	4/3, 5/2	Hopper Transformer Engine; per-tensor scaling

11.6 Compute budgeting (back-of-envelope)

- **Training FLOPs ≈ 6 × N (parameters) × D (tokens).** Inference ≈ **2 × N per token**.
- **MFU** (model FLOPs utilization) = achieved / peak; 35–50% is good for LLM training.
- **Example:** 7B model on 1T tokens = 6 × 7e9 × 1e12 = 4.2e22 FLOPs. One H100 at 40% MFU ≈ 0.4 × 989e12 ≈ 4e14 FLOP/s → ~1.06e8 GPU-seconds ≈ **29,000 GPU-hours** (about 15 days on 80 GPUs). That's how you'd sanity-check a user's allocation request.

11.7 Practical distributed-training checklist (for users)

Seeds and determinism (`torch.manual_seed`; deterministic cuDNN costs speed) · log only from rank 0 · save **sharded/distributed checkpoints** (`torch.distributed.checkpoint`) and resume them · validate on one GPU first, then 1 node, then multi-node · run a **scaling test** · watch for stragglers (one slow node slows everyone) · experiment tracking (TensorBoard, MLflow, W&B in offline mode on nodes without internet).

---

12. LLMs: inference, fine-tuning and "AI-assisted research"

12.1 Inference fundamentals

- **Prefill** (process the prompt; parallel; **compute-bound**) vs **decode** (one token at a time; **memory-bandwidth-bound**).
- **Decode speed ceiling (batch 1):** tokens/s ≈ memory bandwidth ÷ bytes of weights. 7B in BF16 (14 GB) on an H100 (3.35 TB/s) gives ≤ ~240 tokens/s. **Batching** amortizes the weight reads across many requests → throughput rises.
- **KV cache** per token = 2 (K and V) × layers × KV heads × head dim × bytes. Llama-2-7B (32 layers, 32 heads × 128, FP16): 2 × 32 × 32 × 128 × 2 B ≈ **0.5 MB/token** → 4k-token context ≈ 2 GB per sequence. **GQA** (grouped-query attention; e.g. 8 KV heads in Llama-3-8B) cuts that 4×.
- **Serving engines: vLLM** (PagedAttention for KV-cache memory management, continuous batching, OpenAI-compatible API), SGLang, Hugging Face TGI, NVIDIA Triton Inference Server / TensorRT-LLM, llama.cpp/Ollama (CPU and small GPU, GGUF). **Tensor parallelism** for models too big for one GPU (`--tensor-parallel-size 4`).
- **Quantization:** INT8, **4-bit GPTQ/AWQ**, FP8 on H100, GGUF variants; memory ≈ params × bits/8. A 70B model at 4-bit ≈ 35–40 GB.
- **Metrics:** TTFT (time to first token), TPOT (time per output token), throughput (tokens/s across users), p95 latency.
- **Speculative decoding:** a small draft model proposes tokens and the large model verifies several at once → lower latency.
- **Shared-cluster serving design:** a dedicated inference partition or long-running service, a shared read-only **model cache** on fast storage (so users don't each download 140 GB), quotas and rate limits, MIG to slice an H100 into up to 7 instances for small models, and monitoring (Prometheus/Grafana).

12.2 Fine-tuning

- **Full fine-tuning:** all weights; memory as in §11.2.
- **LoRA:** freeze the base and learn low-rank updates $\Delta W = BA$ (rank r, e.g. 8–64) on the attention and MLP projections; trains ~0.1–1% of parameters; adapters are small files.
- **QLoRA:** 4-bit (NF4) frozen base + LoRA adapters in BF16, paged optimizers → a 65–70B model can be fine-tuned on a single 48–80 GB GPU.
- **Tooling:** HF Transformers + **PEFT** + TRL (SFT, DPO), Axolotl, torchtune, Unsloth; DeepSpeed/FSDP for multi-GPU.
- **Instruction tuning / preference tuning** (SFT → DPO/RLHF), and evaluation with held-out sets plus task benchmarks (lm-evaluation-harness).

Your anchor: VarianceEngine. LoRA fine-tuning of **MusicGen-medium** (rank 64, separate learning rates for the new conditioner and the LoRA adapters, BF16, gradient checkpointing) with a custom audio-to-audio cross-attention conditioning head, trained on two GPUs (RTX 4090 + 3090), after comparing five candidate architectures. Be precise about what's finished: you reached a checkpoint (step 1600) with sample outputs; it isn't fully evaluated yet.

12.3 RAG and AI-assisted research

- **RAG pipeline:** ingest documents → chunk (by section, ~300–800 tokens with overlap) → **embed** (e.g. BGE, E5, nomic) → index in a **vector store** (FAISS, Chroma, pgvector, Milvus) → retrieve top-k (hybrid with BM25 keyword search; rerank with a cross-encoder) → prompt the LLM with the retrieved context → cite the sources.
- **When to use which:** prompt engineering (cheapest) → RAG (knowledge that changes, citations needed) → fine-tuning (style, format or domain behaviour; specialized tasks).
- **Agents and tool use:** the LLM calls functions (search, code execution, database queries); coding assistants (e.g. Claude Code) for research-software productivity; notebook copilots.
- **Risks to teach researchers:** hallucination (verify citations), data privacy (don't paste sensitive data into commercial APIs; use local models), reproducibility (pin model versions, log prompts and parameters, temperature 0 for deterministic pipelines), licensing of open weights (Llama community licence, Apache-2.0 for Mistral, Qwen and OLMo variants; check each), evaluation of LLM outputs (human spot-checks, LLM-as-judge with care).
- **Sovereignty (the funding rationale):** run open-weights models on Canadian infrastructure so data never leaves; route regulated data to the **S4H** enclave.

Your anchor: your daily AI-assisted workflow with Claude Code (job-search pipeline, LaTeX CV pipeline, a 14-slide talk on the workflow). "*I can teach AI-assisted research because I practise it, including the verification habits.*"

---

13. Production, monitoring and software engineering for science

13.1 Deploying AI into production (JD line)

- Package the model (TorchScript/`torch.export`, ONNX, or a container with a pinned environment), expose it as a service (REST/gRPC, e.g. FastAPI or an inference server), version models and data, **CI tests** (unit tests plus a numerical regression on a fixed sample), canary or shadow rollout, rollback plan.
- **Monitoring:** system (latency, throughput, GPU memory and utilization, errors) and model (input **data drift**, prediction drift, accuracy against delayed labels). Prometheus + Grafana; DCGM exporter for GPUs; alerts.
- **Efficiency tuning:** batch size, quantization, compile (`torch.compile`, TensorRT), caching, right-sizing the GPU or MIG slice.

Your anchor: MAS care-label QC (scanner → industrial camera → conveyor + PLC reject; inspection time cut ~99.5%), COVID vitals monitoring in hospital wards, MUSMET real-time services under 30 ms. "*I've deployed into environments where failure was physically visible: a reject arm, a hospital ward, a live concert.*"

13.2 Research software engineering habits (Gruner's long-standing concern)

- **Git:** branches, pull requests, meaningful commits, tags for paper versions; never commit data or secrets.
- **Testing:** pytest; numerical tests with tolerances (`np.allclose(a, b, rtol=1e-6)`); regression tests against known-good outputs; CI (GitHub Actions or GitLab CI).
- **Reproducibility:** pin environments (`requirements.txt` with versions, lock files, containers), record seeds, log configs and git hashes with every run, deterministic settings when needed, **FAIR** data, DOIs (Zenodo; you have four datasets there).
- **Workflow managers: Snakemake**, Nextflow, Parsl; Slurm job dependencies for simple pipelines.
- **Documentation:** README, docstrings, a "how to run on SciNet" page.
- **Software Carpentry:** the 2010 Gruner interview was on Greg Wilson's blog (third-bit.com). Wilson founded **Software Carpentry**, and the interview was about SciNet's software-skills course for scientists. Carpentries teaching style: live coding, sticky notes for help, short exercises, formative assessment. Mentioning you know this

will land.

14. Visualization (asset)

- **2D/analysis:** matplotlib, seaborn, plotly (interactive), Bokeh, Altair; dashboards (Streamlit, Dash, Panel).
- **Scientific 3D: ParaView**, VisIt (both built on VTK; client–server mode for remote large data), yt (astrophysics); in-situ visualization (Catalyst, Ascent) to avoid writing huge raw output.
- **ML visualization:** TensorBoard, loss curves, confusion matrices, embeddings (PCA, t-SNE, UMAP), attention maps, saliency (Grad-CAM).
- **Principles:** honest axes, colour-blind-safe palettes (viridis), small multiples, show uncertainty.
- **Your anchor:** d3/topojson interactive globe (Continuity), operational dashboards (COVID vitals, IoT productivity), your graphical-abstract pipeline (Python/CairoSVG).

15. Teaching and training (an asset they listed, and half the role)

- **Design a workshop:** learner persona (e.g. a biology PhD student who knows pandas, not Slurm) → 3–4 concrete learning objectives → hands-on exercises on the Teach cluster or JupyterHub → checkpoints (a quick quiz, sticky notes) → reference page and recording → exit survey → iterate.
- **Explaining to non-IT audiences:** start from their research question, not the tool; one analogy per concept; show a working example first, theory second; avoid jargon or define it once.
- **Scaling the analyst team:** documentation and FAQ pages, office hours, ticket triage (common issues → docs), "recipes" (template Slurm scripts for PyTorch DDP, vLLM serving, HF fine-tuning).
- **Your anchors:** MAS group-wide CV training seminars and hands-on sessions; two BSc theses supervised (2023, 2026); user studies with 20 audience + 6 performers and with musicians; research on pattern recognition as a teaching tool.

16. Numbers worth memorizing

Quantity	Value
Warp size	32 threads
H100 SXM	80 GB HBM3, ~3.35 TB/s, ~989 TFLOPS BF16 dense, 132 SMs, NVLink 900 GB/s
A100 80 GB	~2.0 TB/s, ~312 TFLOPS BF16, 108 SMs, NVLink 600 GB/s
InfiniBand NDR / HDR	400 / 200 Gb/s (50 / 25 GB/s), ~1 μs latency
PCIe Gen5 x16	~64 GB/s per direction
Cache line	64 bytes
DRAM latency	~100 ns; L1 ~1 ns
Adam mixed-precision memory	~16 bytes/param
Inference weights	2 bytes/param BF16; 0.5 bytes/param 4-bit
Training compute	≈ 6·N·D FLOPs
KV cache Llama-2-7B FP16	~0.5 MB/token

Quantity	Value
Amdahl, p = 0.95	max speedup 20×
Trillium	241,056 cores, 252 GPUs; launched 2025-08-07
New AI system	$42.5M; announced 2025-11-28

17. Scenario drills (practise aloud: diagnosis → fix → prevention)

1. **"My 4-GPU job is barely faster than 1 GPU."** → Check GPU utilization; dataloader workers and pinned memory; small-file I/O (shard or stage locally); per-GPU batch too small; `DataParallel` instead of DDP; NCCL falling back to TCP; rank-0-only work (validation, logging). → Profile one step with `nsys` / `torch.profiler`. → Prevention: a recipe page + template script.
2. **"CUDA out of memory."** → `nvidia-smi` for other processes; memory ladder (§11.2); check for memory leaks (keeping the graph via `loss` accumulation without `.item()`, storing tensors in lists).
3. **"My job has been pending for two days."** → `squeue --start`, `scontrol show job` (Reason: Priority/Resources/QOS); requested wall time too long (backfill); too many GPUs requested; fair-share used up → advise a smaller or shorter job, or a RAC allocation next cycle.
4. **"Job killed after 24 h and I lost everything."** → Wall-time limit; teach checkpoint/resume + `--dependency=afterok` chaining; use `--signal=USR1@300` to checkpoint before the kill.
5. **"pip install fails on the compute node."** → No internet on compute nodes; build the venv on a login node with the cluster wheels; `--no-index`; or use an Apptainer image.
6. **"My MPI code hangs at 64 ranks but works at 8."** → Deadlock from message size crossing the eager/rendezvous threshold (blocking send-send); mismatched collectives; one rank errored silently. → `gdb` attach, rank-tagged logs, `MPI_Sendrecv` /non-blocking.
7. **"Results differ between runs/GPUs."** → Floating-point non-associativity in parallel reductions, non-deterministic cuDNN/atomics, TF32 enabled, different library versions → set seeds, deterministic flags, pin versions; judge differences against a tolerance, not bitwise.
8. **"The filesystem is slow for everyone" (admin view).** → One user's job opening millions of small files or writing checkpoints every step → contact the user, teach sharding and checkpoint cadence.
9. **"A group wants a private ChatGPT over their papers."** → vLLM serving an open-weights model on the AI system + RAG (embeddings + FAISS) + citation display; sensitive data → S4H; eval set of known Q&A; licence check.
10. **"A climate modeller wants to try ML on their Fortran simulation output."** → Read NetCDF with xarray; chunk with Dask/Zarr; start with a baseline (linear or random forest), then a CNN/U-Net or neural-operator surrogate; group or temporal split to avoid leakage; possibly couple back into Fortran via ONNX Runtime or a Python bridge.
11. **"Port this Python loop to be fast."** → NumPy vectorization → Numba → CuPy if it's big and data-parallel; always verify numerics.
12. **"How would you decide how many GPUs a fine-tuning job needs?"** → Model size × bytes/param (LoRA vs full), activation estimate at the target sequence length and batch, then 6·N·D for time; pick the smallest config that fits, then scale-test.

18. Self-quiz (cover the answers, then check)

1. What's the difference between a login node and a compute node? → *Login: edit, compile, submit, no heavy compute; compute: scheduler-allocated, often no internet.*

2. Why do shorter wall-time requests often start sooner? → *Backfill fills scheduling gaps with short jobs.*
3. Apptainer vs Docker on HPC? → *Rootless, single SIF file, `--nv` for GPUs; Docker needs a root daemon.*
4. Amdahl's maximum speedup for 98% parallel code? → *1/0.02 = 50×.*
5. Strong vs weak scaling? → *Fixed total problem vs fixed problem per worker.*
6. Two causes of OpenMP slowdown besides too few threads? → *False sharing; NUMA/remote memory (no first-touch or pinning); load imbalance; critical sections.*
7. Classic MPI deadlock? → *Both ranks do a blocking Send first; fix with Sendrecv or non-blocking calls.*
8. What does all-reduce decompose into? → *Reduce-scatter + all-gather.*
9. Where does tensor parallelism belong and why? → *Inside a node on NVLink; communication-heavy per layer.*
10. Memory to fully fine-tune 7B with Adam in mixed precision? → *~112 GB plus activations → FSDP over ≥2 × 80 GB GPUs, or use LoRA.*
11. BF16 vs FP16? → *Same 8-bit exponent as FP32 (range) vs 5-bit; FP16 needs loss scaling.*
12. Why is LLM decode memory-bound? → *Each token reads all the weights for very few FLOPs; low arithmetic intensity.*
13. KV cache size formula? → *2 × layers × KV heads × head dim × bytes per token.*
14. Roofline ridge point meaning? → *The arithmetic intensity where a kernel moves from memory-bound to compute-bound.*
15. Row-major vs column-major: which loop goes inside in Fortran? → *The first index (rows).*
16. What does `intent(inout)` do? → *Declares the argument is read and modified; the compiler checks usage.*
17. Why is conda discouraged on clusters? → *Huge numbers of small files (quota, metadata load) and library conflicts with the system MPI/CUDA.*
18. Leakage example in a medical ML project? → *Same patient in train and test; normalization fit on the full data.*
19. PR-AUC vs ROC-AUC? → *PR-AUC is more informative for heavy class imbalance.*
20. 400 Gb/s in GB/s? → *50 GB/s.*
21. What is GPUDirect RDMA? → *NIC ↔ GPU memory transfer without staging through host RAM.*
22. Training FLOPs for 1B params × 100B tokens? → *6 × 1e9 × 1e11 = 6e20.*
23. How does QLoRA fit a 70B model on one GPU? → *4-bit frozen base (~35–40 GB) + small BF16 adapters + paged optimizer.*
24. What does `seff` tell you? → *CPU and memory efficiency of a finished job (used vs requested).*
25. First thing to check when GPU utilization is 20%? → *The input pipeline: dataloader workers, I/O, CPU decode.*

---

19. Honest-gap language (keep it ready)

- **Fortran / MPI / Slurm (if still thin by interview day):** "*I haven't used it in production. Here's exactly what I've done hands-on in the last [N] days, and here's how I'd ramp up: SciNet's own courses and pairing with the analysts who've run MPI codes for years. My strength for this seat is the GPU, PyTorch, LLM and data side the new system needs.*"
- **Multi-node training at scale:** "*My largest run was two GPUs. But the concepts carry over (DDP, FSDP, NCCL, scaling tests), and my career has been about extracting performance under hard constraints. I'd validate everything on Trillium's GPU nodes before advising users at scale.*"
- Never bluff a number or a tool. A wrong confident claim to a panel of physicists costs more than an honest "not yet."

---

20. Sources and further reading

- SciNet: https://scinethpc.ca/about-scinet · Trillium: https://scinethpc.ca/trillium/ · New AI system: https://scinethpc.ca/?p=5083
- SciNet user docs (Trillium, Slurm, modules, filesystems): https://docs.scinet.utoronto.ca
- SciNet education/courses: https://education.scinet.utoronto.ca
- Digital Research Alliance of Canada docs (Python, Apptainer, Slurm, PyTorch multi-GPU): https://docs.alliancecan.ca
- PyTorch DDP / FSDP tutorials: https://pytorch.org/tutorials/
- NVIDIA H100 and A100 datasheets (specs in §2.2): https://www.nvidia.com/en-us/data-center/h100/ · https://www.nvidia.com/en-us/data-center/a100/
- vLLM docs: https://docs.vllm.ai
- Hugging Face PEFT (LoRA/QLoRA): https://huggingface.co/docs/peft
- MPI tutorial: https://mpitutorial.com · OpenMP reference: https://www.openmp.org/resources/refguides/
- Software Carpentry lessons: https://software-carpentry.org/lessons/
- Gruner interview (2010): https://third-bit.com/2010/07/08/interview-scinet/